

COMP8640 Artificial Intelligence for Natural Language Processing Applications

Practical Use of AI for Advanced NLP Applications

Business problem:

Problem definition:

- Enterprises get a lot of unstructured consumer input in the form of reviews, conversations, and emails. Compliance groups need to identify hazards associated with the business (refund processing, delivery conflicts, privacy/security issues), enumerate them for action, respond with advice that complies with the policy, and gain knowledge from user comments. Manual processing is slow, erratic, and prone to mistakes. We need an automated assistant that classifies issues, retrieves relevant policies/SOPs, and generates concise, grounded answers.
- LLMs (large language models) are beneficial in three ways. To improve prioritizing and routing, classification models first sort each message into the following categories: billing/refunds, delivery/logistics, support responsiveness, security/fraud, privacy, and praise. Second, Retrieval-Augmented Generation (RAG) reduces hallucinations by ensuring that responses are precise and specific by grounding them in our written policies and SOPs. Third, a feedback loop records user comments and "usefulness," which leads to immediate modification, more seed instances for the conventional classifier, and improved retrieval coverage over time. Faster, policy-correct replies, less compliance risk, and enhanced customer satisfaction are the outcomes.

System architecture & workflow:

Components:

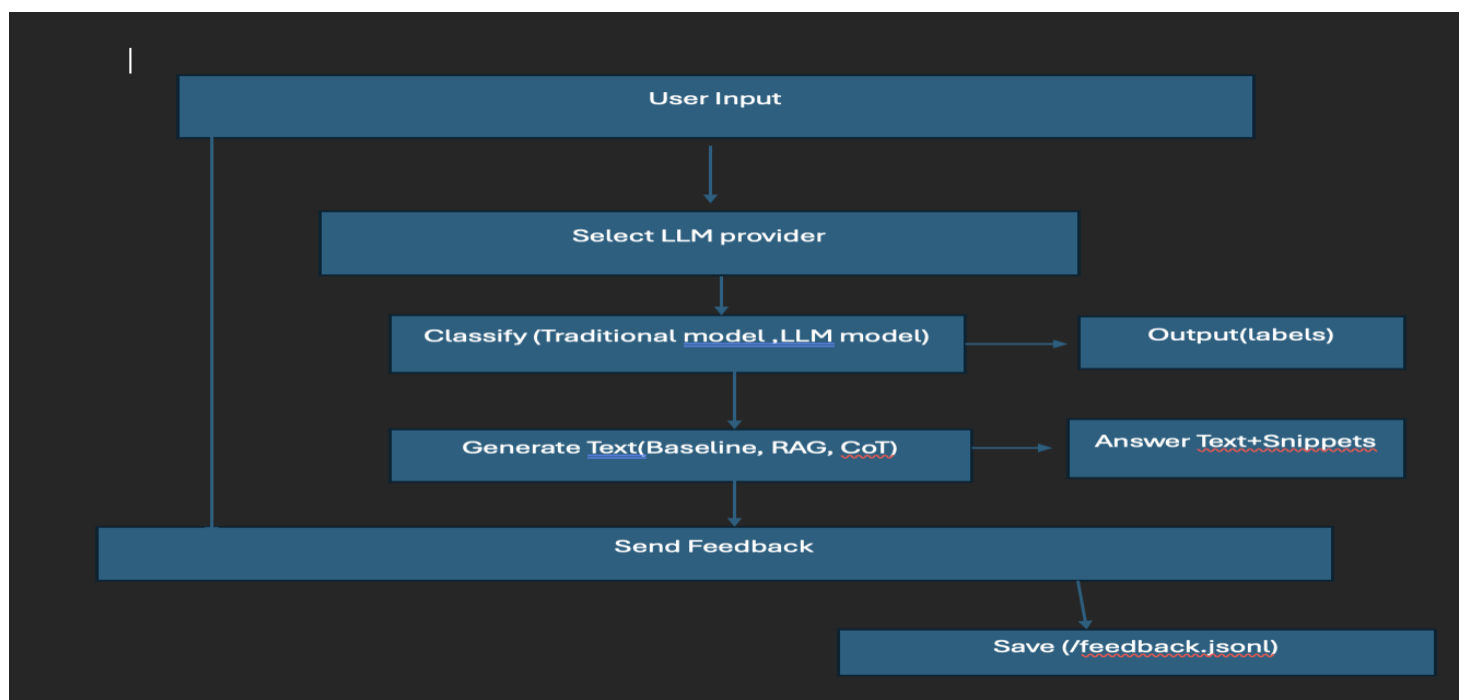
- Frontend (streamlit web app): The user can write a review, complaint, or question into a single input form.
- Classify: identifies the type of problem (e.g., Delivery/Logistics, Privacy, Billing/Refunds, etc.).
Generate Answer: the function generates a brief, policy-based response that includes distinct citations to the supporting documentation.
- Feedback: The user can comment and rate usefulness using a feedback panel. Future enhancements to prompts, training data, and model selection are guided by the feedback, which is saved in JSONL format.

- Sidebar controls: Choose between Baseline, RAG, and CoT generation using the answer mode selector.
- Classifier selector: alternate between an LLM-based classifier (zero-shot) and the traditional (TF-IDF + Logistic Regression) model.
- LLM provider/model: select the particular model to execute and the active LLM backend (such as Hugging Face or local LLM (ollama) or open AI).

Experience of the user: The generated response, the projected label, and references to the company's grounding rules and SOPs are shown right beneath the input. To facilitate continual development without interfering with the user interface, the feedback flow closes the loop by collecting user evaluations, which we then analyze to improve prompts, increase training seeds, and modify retrieval settings.

Data flow:

The data flow is simple: a user submits a complaint, review, or question through the front end, which forwards the request to the backend. The backend starts by classifying the input to assign a category (such as Billing/Refunds, Delivery/Logistics, Support Responsiveness, Security/Fraud, or Privacy). In RAG or CoT modes, it subsequently retrieves the top-k (k value=3) relevant policy snippets, refines them, and constructs a prompt incorporating the user's inquiry. Utilizing deterministic decoding, the model produces a clear, well-founded response. The backend sends back the final response along with individual citations, which the UI presents to the user. Lastly, any feedback from the user is recorded and stored for analysis and to improve future prompts, retrieval settings, or training data.



Frontend:

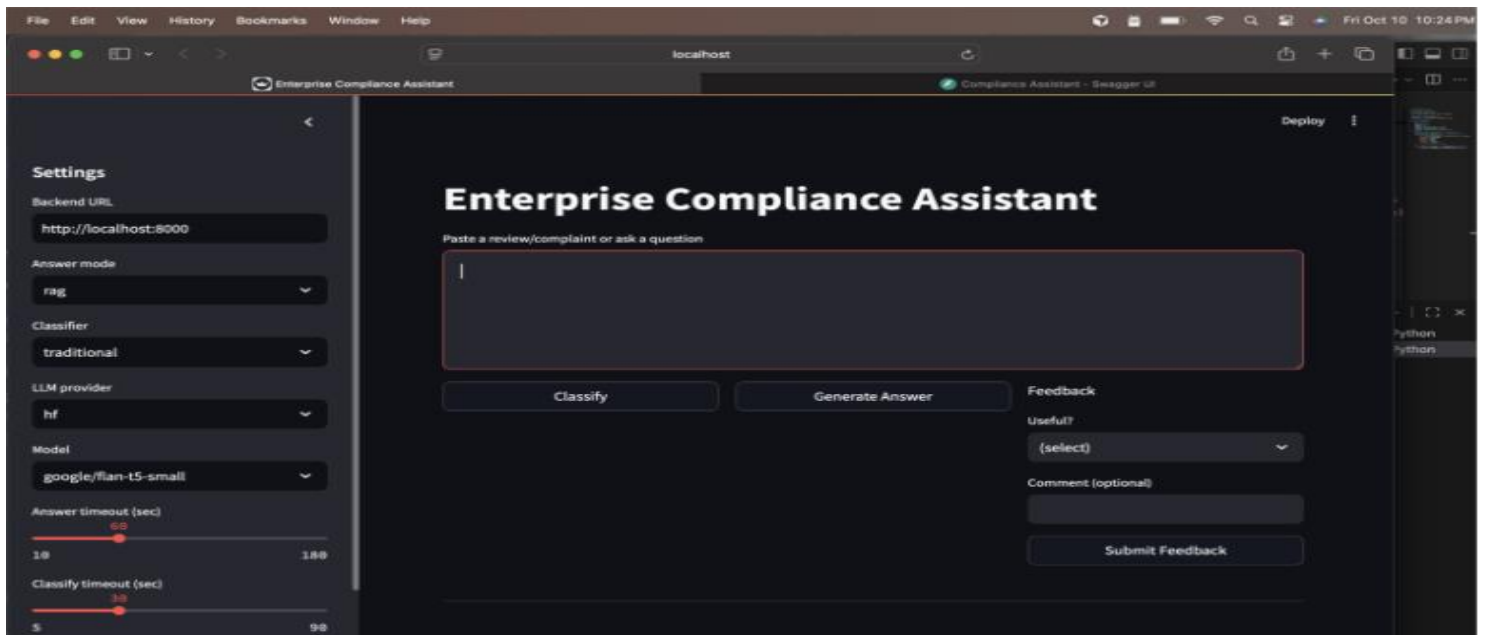


Fig 1: the whole frontend page

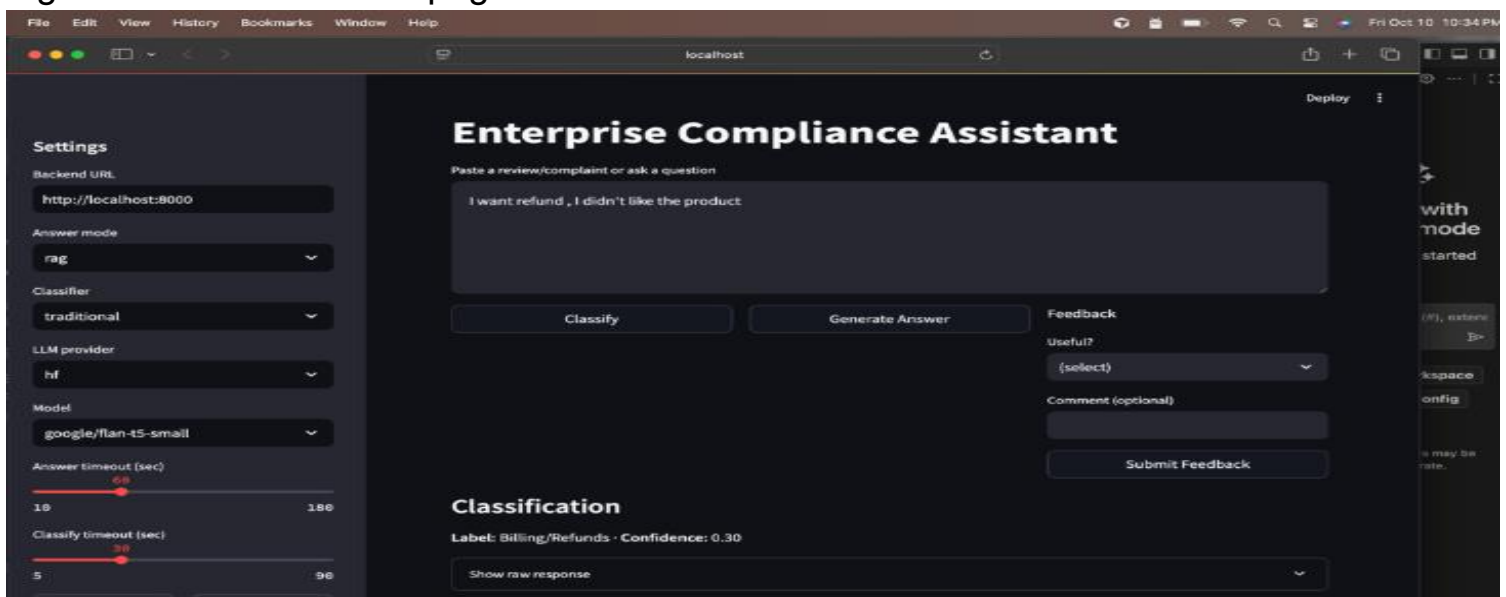


Fig 2: Enter the user input and classify using traditional classification and we got the label :billings/refund

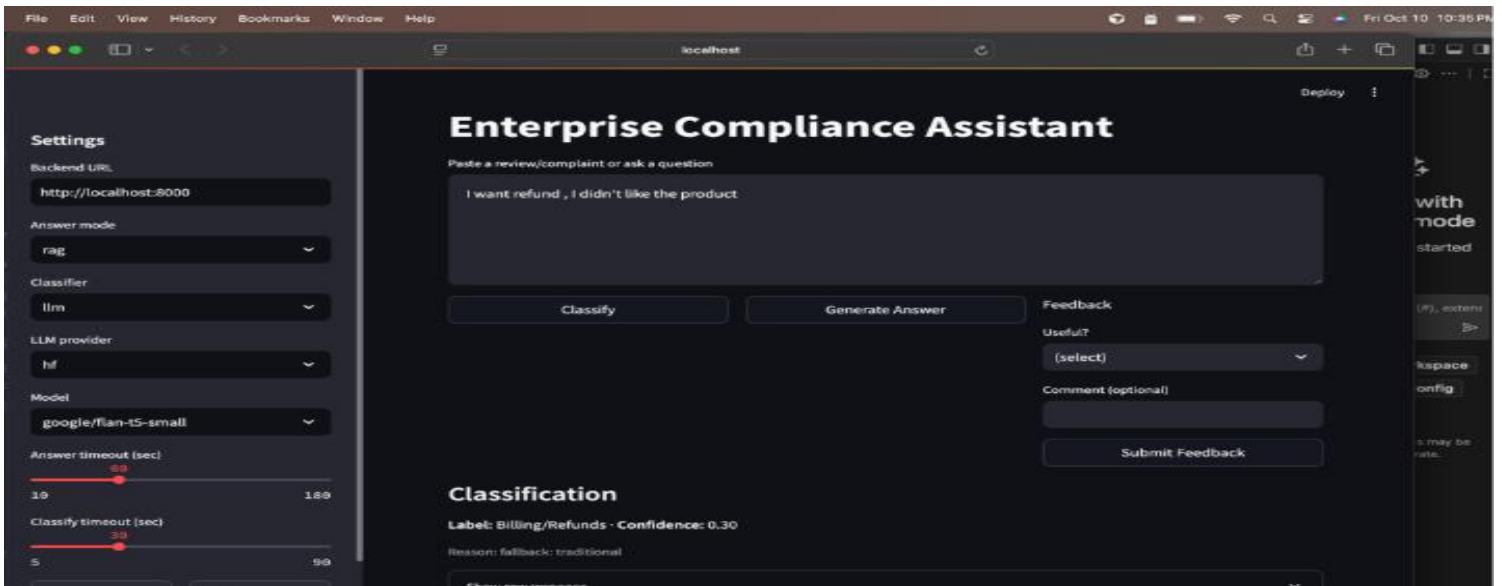


Fig 3: Classifying using LLM and got the same label

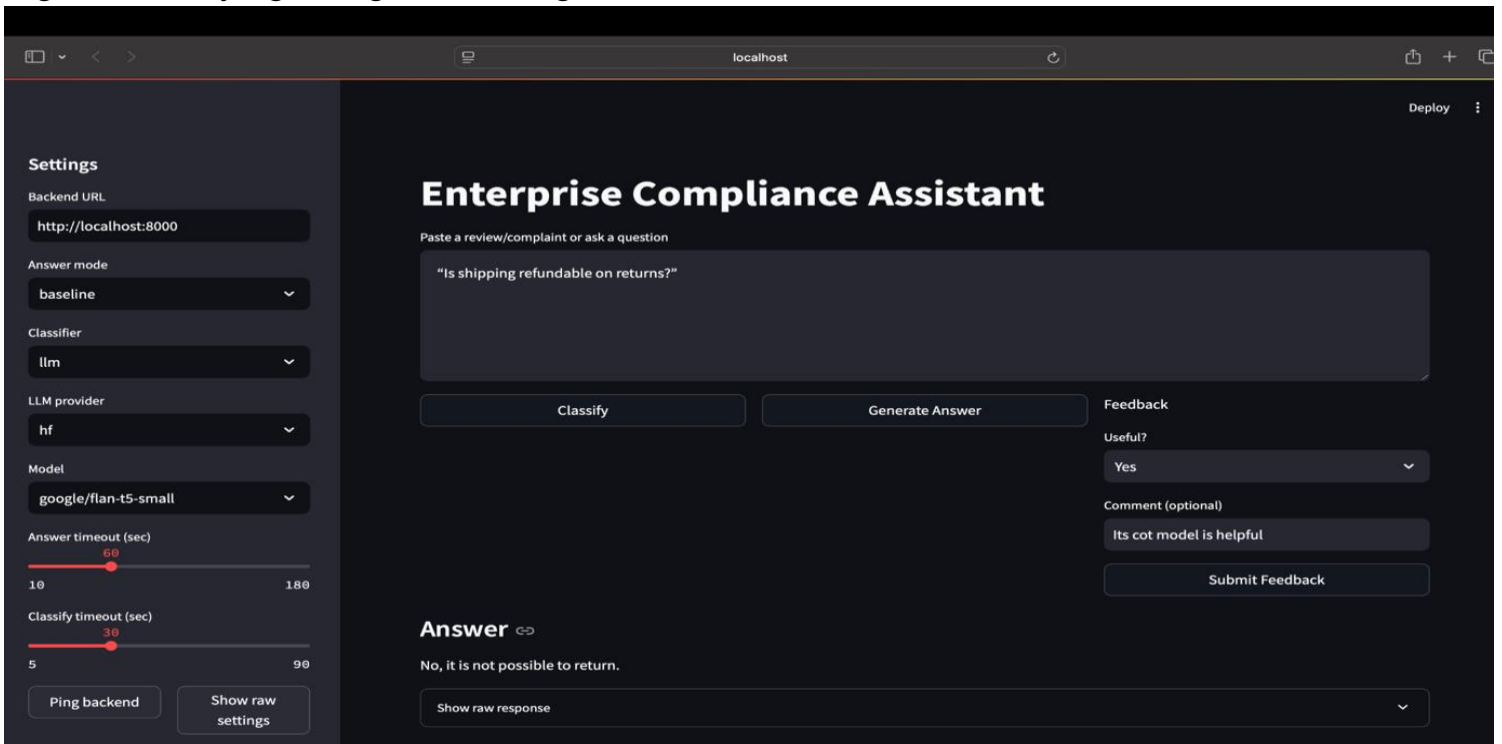


Fig 4: Generate answer using Baseline model

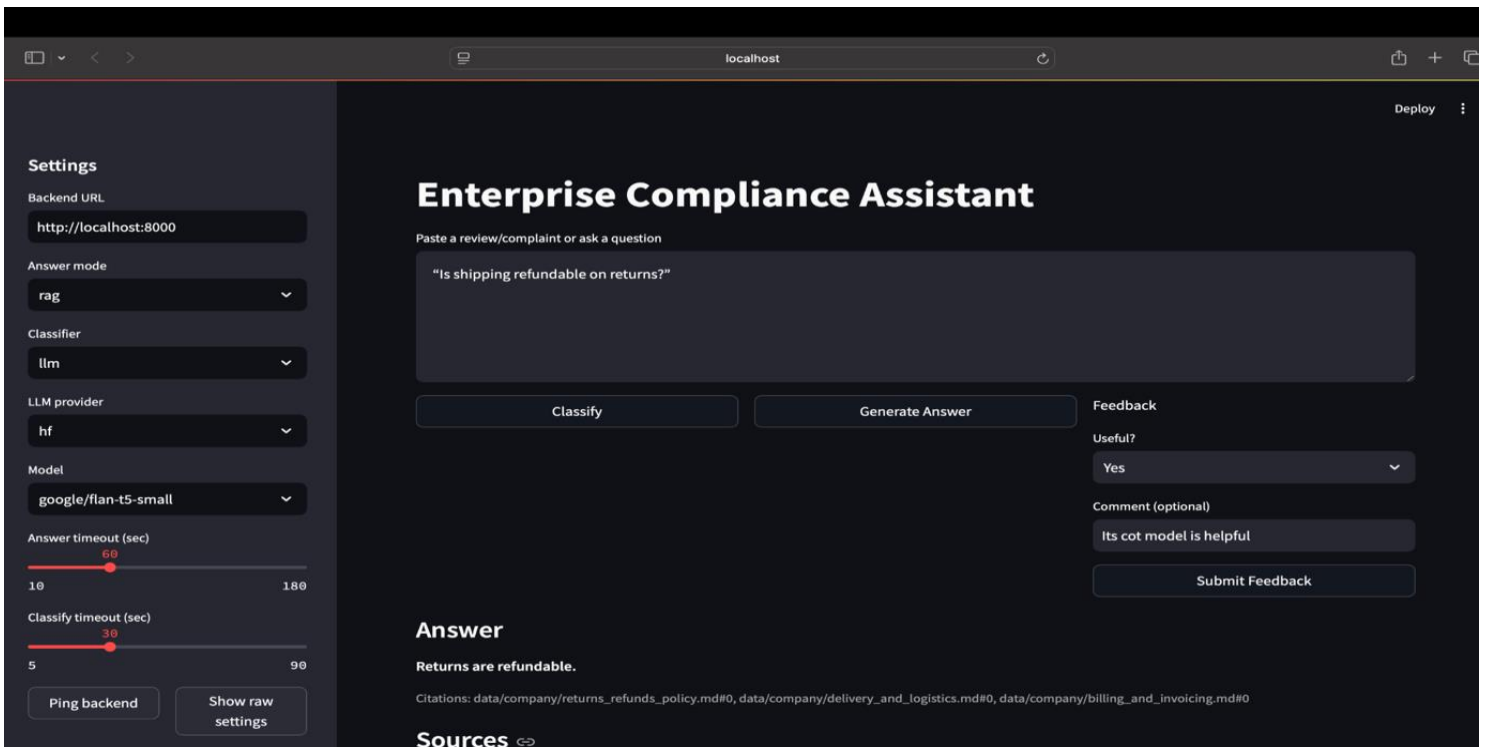


Fig 5 : Generated answer using RAG model

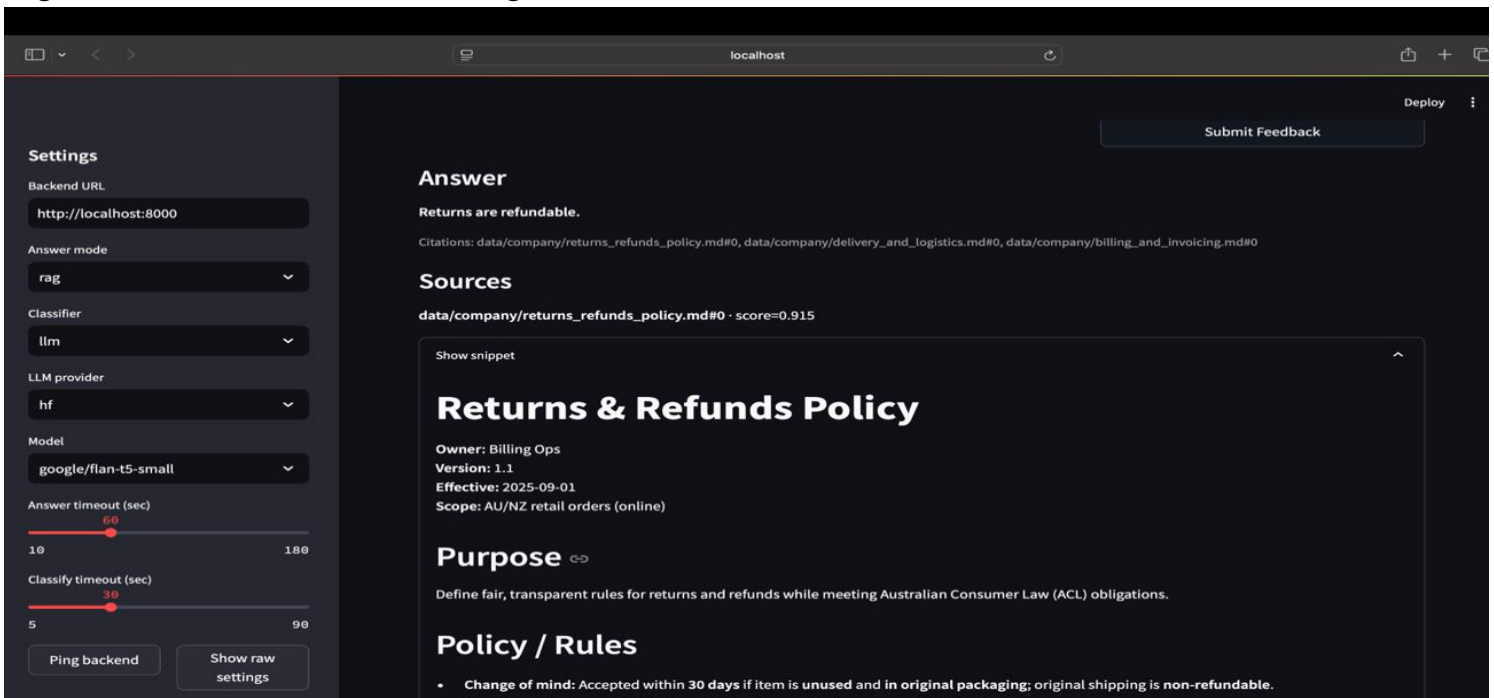


Fig 6: RAG model fetched the top 3 snippets related to the query.

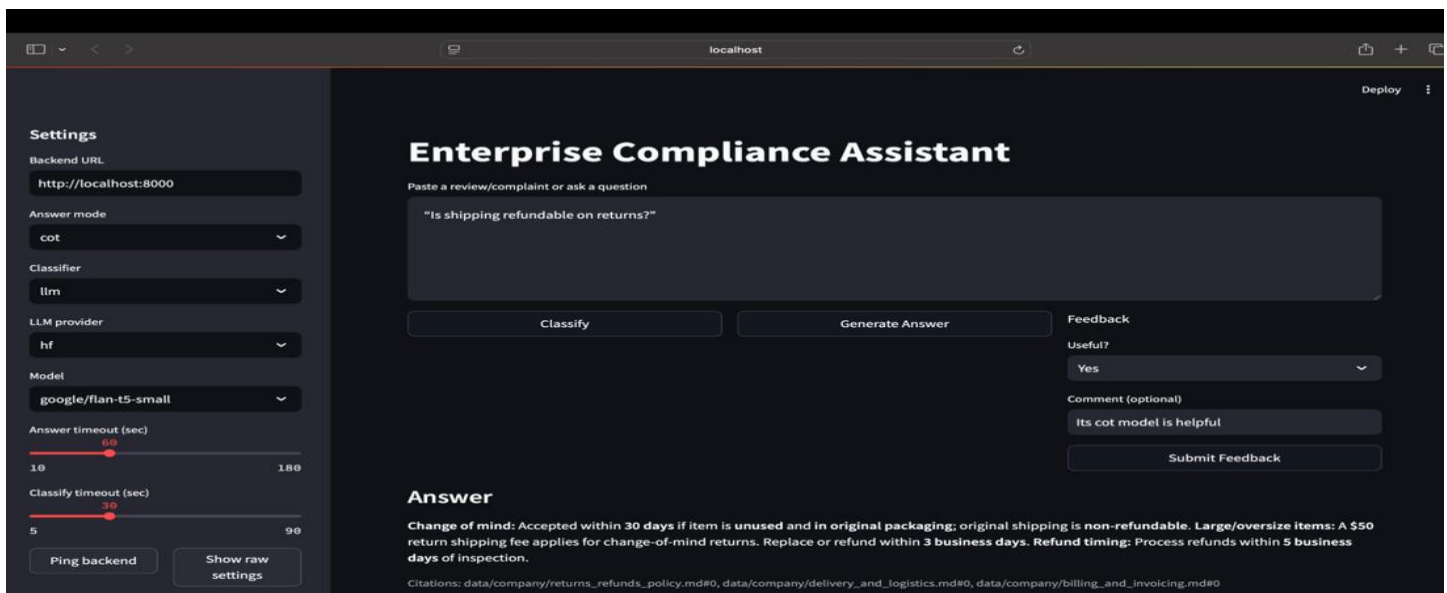


Fig 7: Generated answer using CoT got the answer.

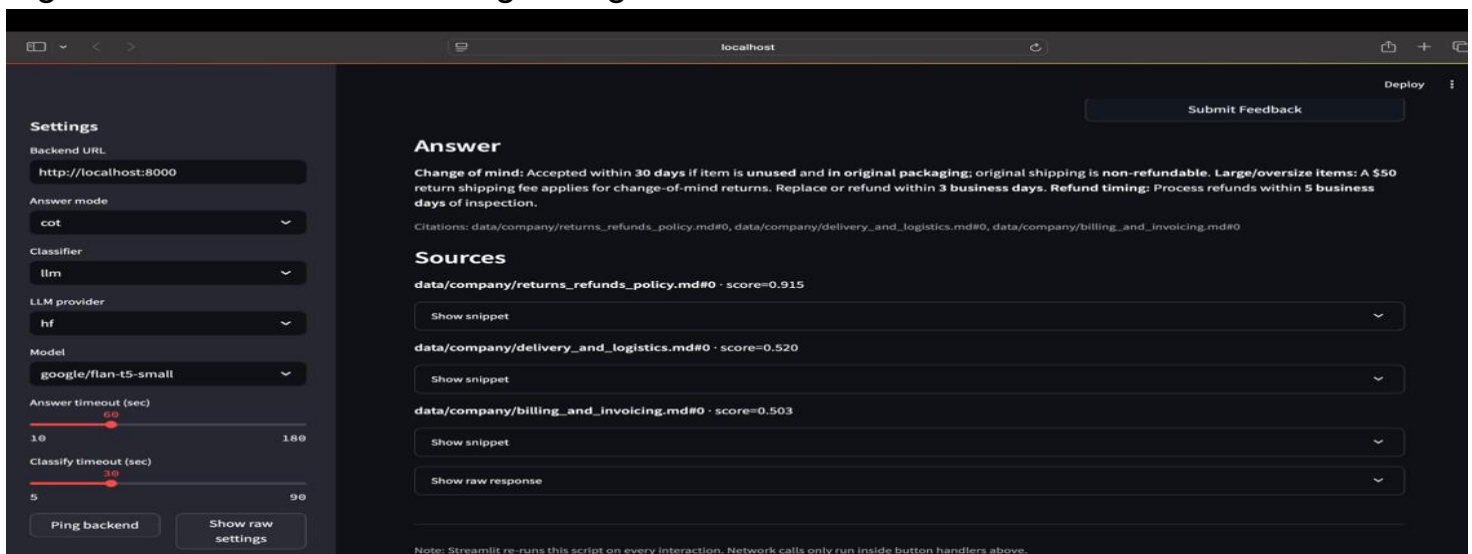


Fig 8: CoT model fetched the top 3 snippets related to the query.

Figures 5–8 demonstrate our use of Retrieval-Augmented Generation (RAG) and Chain-of-Thought (CoT) prompting. For each user query or customer comment, the system retrieves the most relevant internal policies and SOPs, then generates a concise, customer-facing answer grounded in those sources. CoT improves reasoning clarity for multi-step or ambiguous cases (e.g., differentiating change-of-mind returns from ACL-covered faults), while RAG ensures the response cites the correct documents. Together, these techniques consistently align answers with company policy and reduce hallucinations, as evidenced by the policy citations shown alongside each generated response in Figures 5–8.

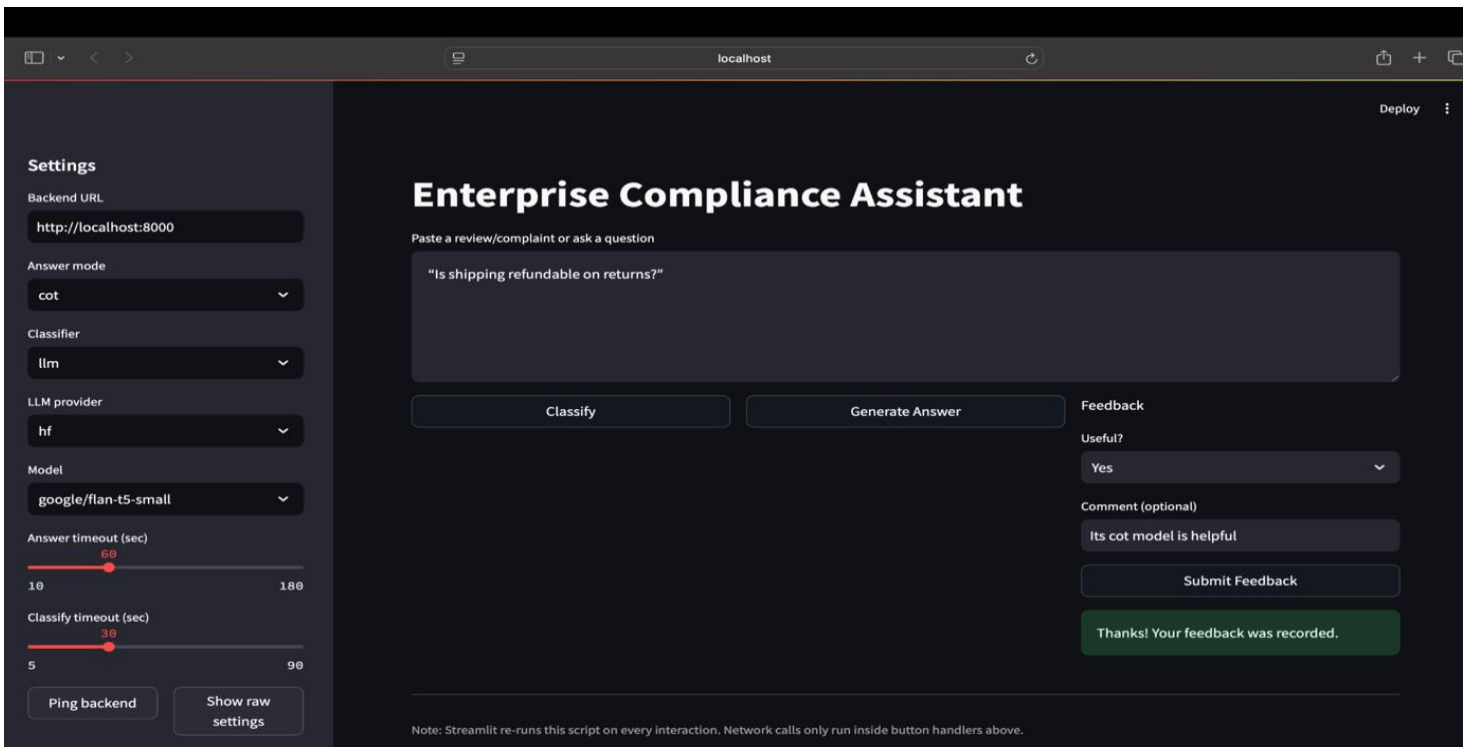


Fig 9 We can send a Feedback saying where that's used full or not with a comment.

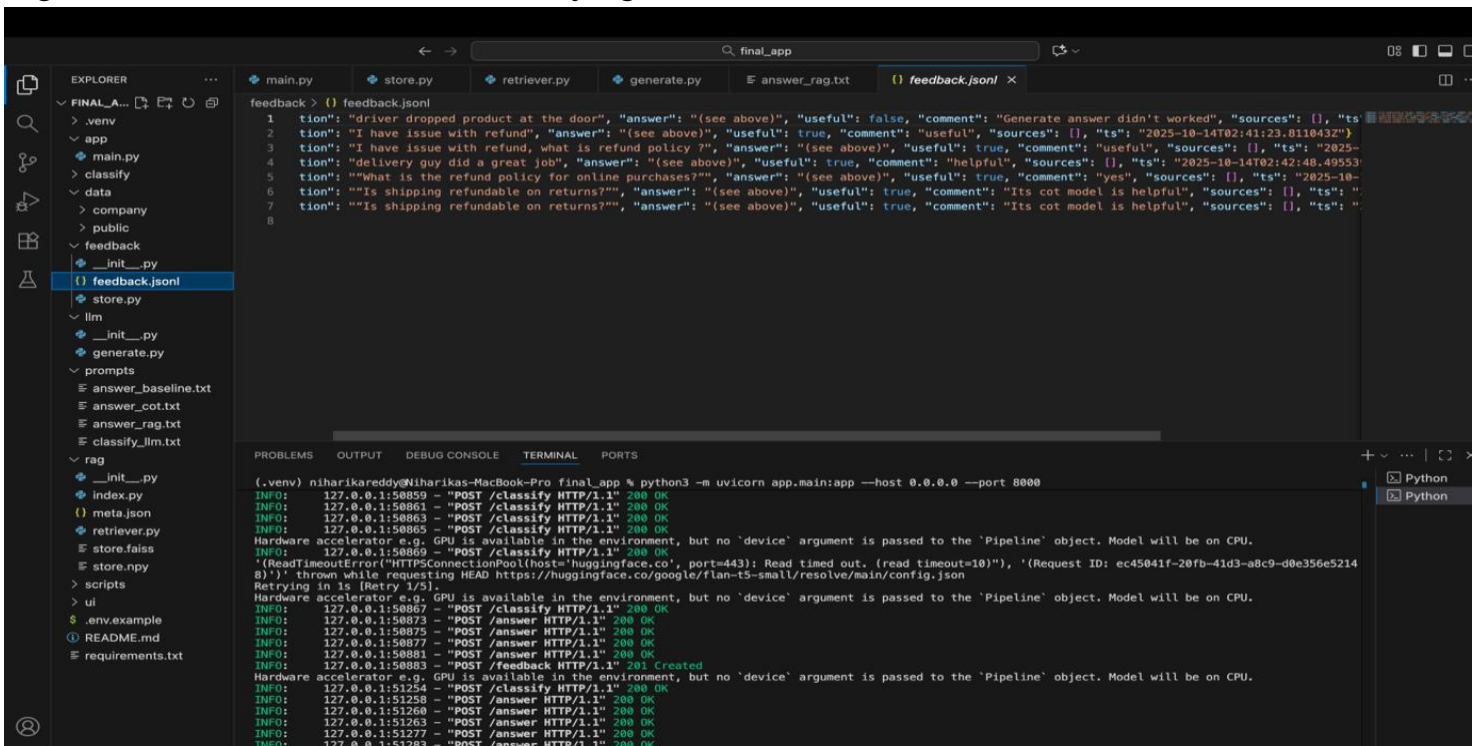


Fig 10: Feedback is stored in /feedback/seed.jsonl file for further training.

LLM interaction & response demonstration:

Collect company information:

- We authored a set of original, company-specific policies and SOPs in data/company/ to ground the system's answers, including returns_refunds_policy.md, delivery_and_logistics.md, data_retention_policy.md, data_deletion_sop.md, and security_incident_response_sop.md (among others). To create a realistic customer context, we developed initial review excerpts for retrieval in data/public/. The entire dataset was subsequently indexed using python3 -m rag.index, allowing the RAG pipeline to pull the most pertinent policy excerpts and reference them alongside the generated replies.

Classify text:

- For our classification task, we assessed two methods. The conventional approach utilizes TF-IDF features (1–2 grams) in conjunction with a Logistic Regression classifier (class_weight='balanced') to address class imbalance issues. Our main label set encompasses Billing/Refunds, Delivery/Logistics, Support Responsiveness, Security/Fraud, and Praise, while Privacy and Product Quality were included in exploratory analyses. Additionally, we explored an LLM-based classifier as an alternative option, limited by a JSON-only prompt and a temperature setting of 0 to ensure deterministic results; this method also features a comprehensive JSON parsing fallback along with basic keyword heuristics to prevent null classifications.

Generate textual responses:

- Baseline: In Baseline mode, the model generates responses solely based on the user's input and a brief instruction prompt. It operates quickly and smoothly, but lacks access to company policies or previous snippets, which can result in generic replies (for example, "You can ask for a refund within 30 days") and might overlook specific rules or fees related to edge cases.
- RAG: RAG (Retrieval-Augmented Generation) initially searches our curated knowledge database, which includes company information and vetted public reviews, to extract the most relevant top-k passages—such as returns policies, delivery standard operating procedures (SOPs), and privacy/deletion guidelines. The model then formulates the answer based solely on those excerpts and provides separate citations to the exact documents referenced. For instance, if a

customer queries, “Can I exchange an item?”, RAG retrieves the “Returns & Refunds Policy” and the “Delivery & Logistics” SOP, and subsequently generates a fact-based response (including details like the exchange period, item condition requirements, and postage responsibilities), accompanied by citations such as data/company/returns_refunds_policy.md#0. This approach significantly minimizes inaccuracies and guarantees alignment with established policies.

- CoT: The same retrieved fragments are used in CoT (Chain-of-Thought) with RAG, but a quick concealed reasoning step is added prior to the final message. This enables the model to process situations with multiple steps (such as "change-of-mind vs faulty item," "oversized item fees," or "ACL obligations") and then provide a clear response that the client can see. When faced with unclear questions, such as "I didn't like it—can I return it and who pays shipping?" CoT is most useful in practice since it balances the rules and returns the appropriate approach (eligibility, costs, timescale) while rooting in the policy in question.

LLM and Advance Techniques: Below is the table which listed all the techniques used.

LLM Technique	Model	What it is	How we used it in this assignment
Prompting (Baseline)	google/flan-t5-small (Hugging Face, text2text)	Small instruction-tuned LLM; deterministic with temperature=0.	Answers user questions without retrieval using prompts/answer_baseline.txt (concise, no IDs/paths).
RAG – Retrieval	sentence-transformers/all-MiniLM-L6-v2 + FAISS	Embedding model for semantic search; FAISS for vector index.	Indexes data/company/* and data/public/*; retrieves top-k snippets feeding prompts/answer_rag.txt.
RAG – Generation	google/flan-t5-small	Grounded generation constrained to retrieved snippets.	Composes prompt from user question + cleaned snippets; returns answer first, citations separately.
Chain-of-Thought (CoT)	google/flan-t5-small	Brief hidden reasoning step	Uses prompts/answer_cot.txt; clarifies multi-condition cases (e.g.,

		before the final answer.	exchange vs faulty) with RAG snippets.
Traditional Classifier	TF-IDF (1–2 grams) + Logistic	Classic supervised text classifier; transparent and fast.	Predicts labels (Billing/Refunds, Delivery/Logistics, Support Responsiveness, Security/Fraud, Praise); trained from classify/seed.jsonl.
LLM Classifier	Zero shot	Prompted zero-shot classification with structured JSON output.	Classifies when selected; deterministic decoding; robust JSON parse with keyword fallbacks.
Refinement / Fine-tuning path	Feedback loop + retrain TF-IDF/LR;	Data-driven improvement of prompts and traditional model; future path to light LLM tuning.	Logs feedback to feedback/feedback.jsonl; adds seed examples; retrains classifier; leaves option for LoRA if permitted.

Result and Evaluation:

Testcase1:

Question/Review: I was charged twice for the same order—how do I get a refund?

Label: Billing/Refunds · Confidence: 0.33

Generated answer:

Baseline: I got charged twice for the same order.

RAG: Refund timing: Process refunds within 5 business days of inspection. Change of mind: Accepted within 30 days if item is unused and in original packaging; original shipping is non-refundable. Duplicate charges are automatically detected; suspected duplicates are refunded within 3 business days.

CoT:

Testcase2:

Question/Review: I got a password reset email I didn't request—what should I do?

Label: Label: Security/Fraud · Confidence: 0.32

Generated answer:

Baseline: I don't want to use the email.

RAG: If suspicious activity → force password reset and 2FA prompt. Contain: force sign-out on all devices; recommend password reset + enable MFA/passkey. For copyright or scam reports, I can direct you to the right form." Unknown

charges / Account on hold / Protect Your System Identify real Amazon communications (email/phone/SMS/web) Manage your personal information Trustworthy Shopping at Amazon Do not request sensitive credentials or full card numbers. Password minimum: 10 chars; support for passkeys.
CoT: Do not request sensitive credentials or full card numbers. Avoid sharing internal tooling or investigation details with reporters. Keep interaction logs minimal and factual; include timestamps and links. Escalate immediately if there is evidence of active compromise affecting multiple users.

Testcase3:

Question: Tracking says delivered but nothing is at my door—what happens next

Label: Delivery/Logistics · Confidence: 0.29

Generated answer:

Baseline: I'm not sure what the package is delivered to.

RAG: return policy / cross-border returns cost Suggested route: Billing/Refunds;

Returns policy clarification Item was not delivered... driver took a photo BUT the \$90 [item] was not included. Replace or refund within 3 business days. Standard delivery: 2–6 business days metro; 3–10 regional. If unresolved after 5 business days, escalate to Complaints Manager.

CoT: Replace or refund within 3 business days. Theme: return policy / cross-border returns cost Suggested route: Billing/Refunds; Returns policy clarification Item was not delivered... driver took a photo BUT the \$90 [item] was not included. If unresolved after 5 business days, escalate to Complaints Manager. Acknowledge within 24 hrs . provide case ID.

Testcase4:

Question : I got the product and its really nice

Label: Label: Praise · Confidence: 0.24

Note: Since we didn't add anything related to the Parise in data/company.md file it fetched top 3 policies and details about refund.

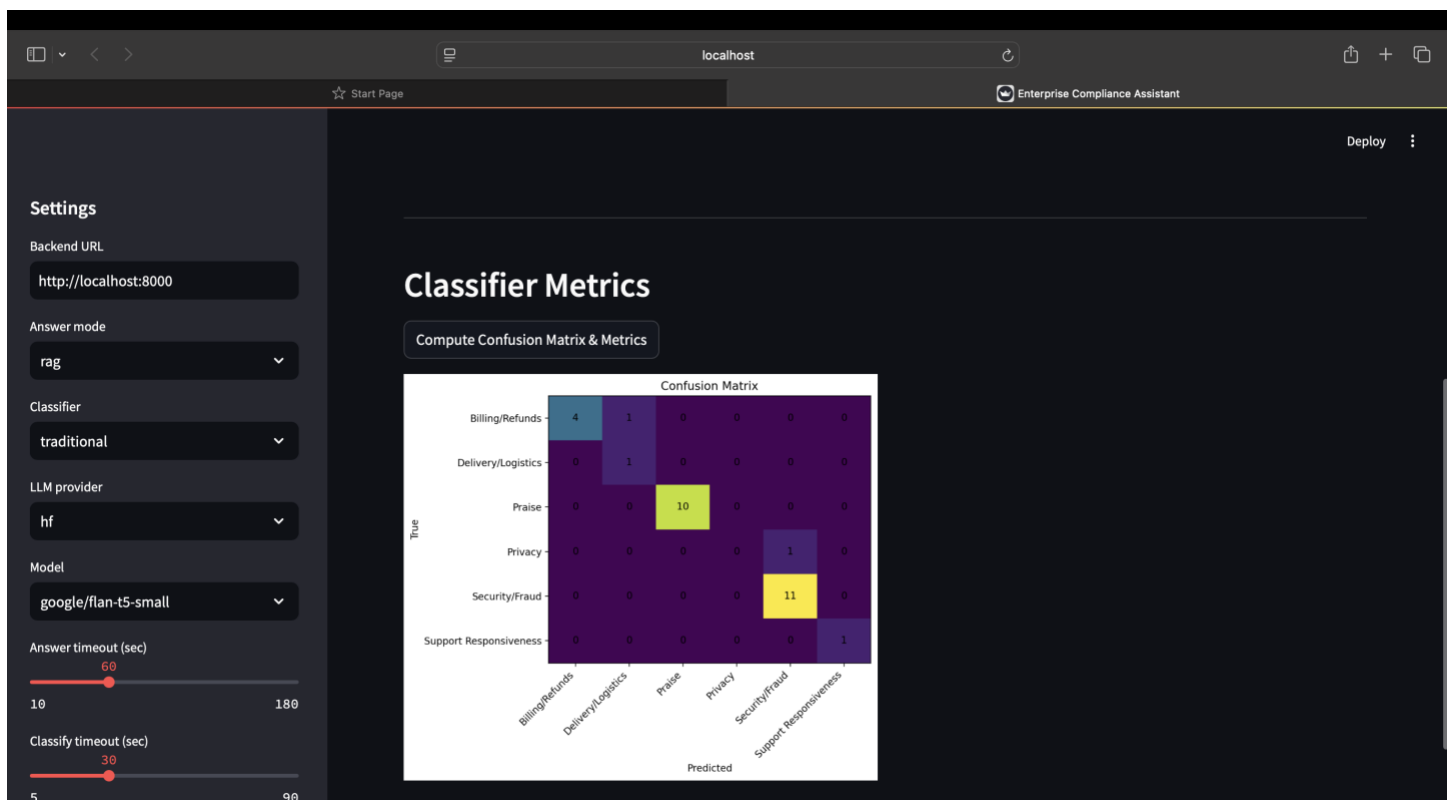
Drawbacks:

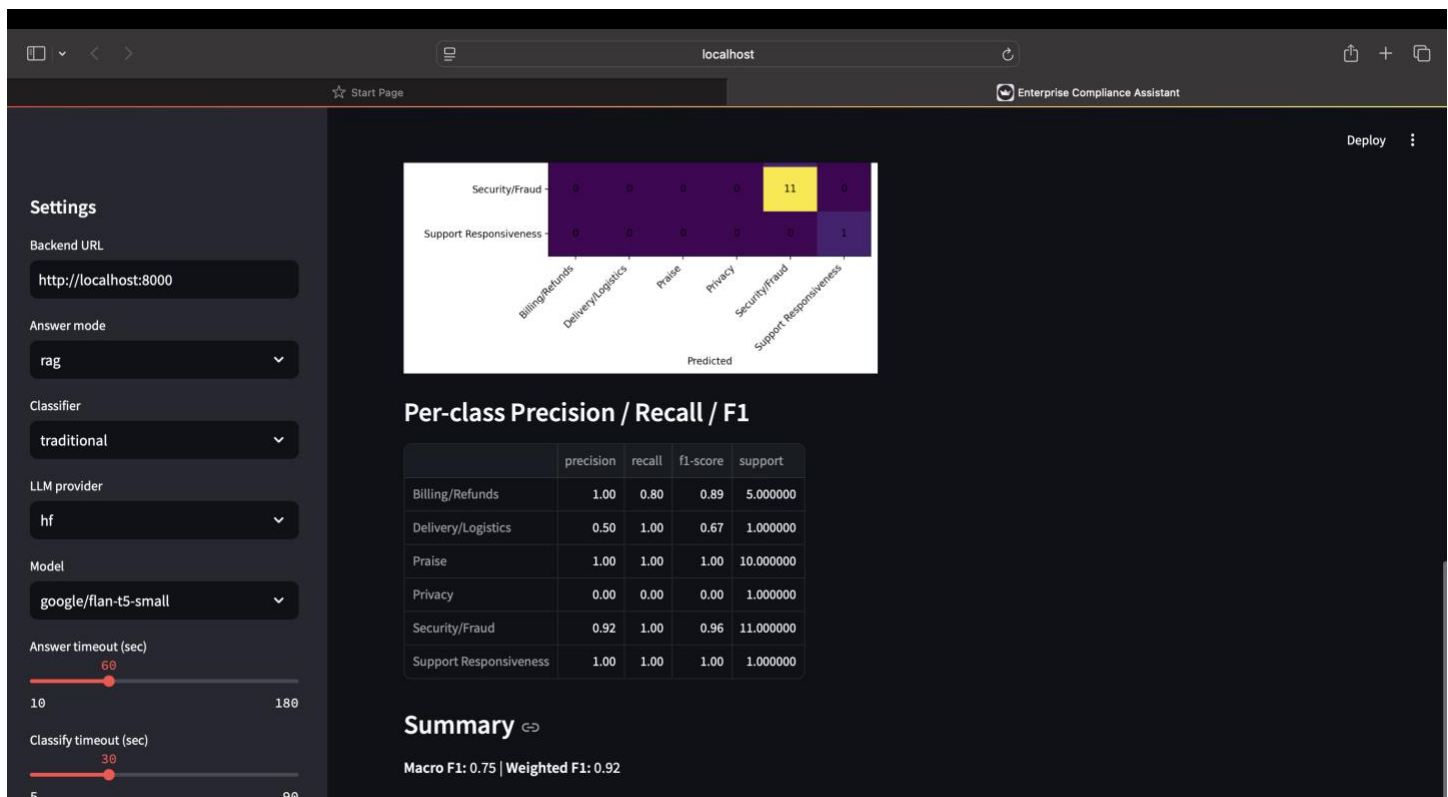
- During testing, some text examples failed due to classification inconsistencies, specifically between the security/fraud, privacy, and support response categories. Although the model accurately identifies positive "praise" inputs, it often generates irrelevant results because there are no linked reference files for that class. As a result, the system retrieves unnecessary compliance or refund data. The training dataset currently consists of only a small selection of Amazon reviews and policy documents, a few public websites, and open datasets. Increasing and changing the training data would significantly improve accuracy and contextual awareness.
- The integration of publicly accessible datasets, like those from Hugging Face (HF) or Kaggle, could greatly improve model resilience in subsequent developments. Despite being included in the scripts, these datasets were not included in the

current phase since the results were not trustworthy enough. Incorporating them into the subsequent version may enhance answer accuracy, diversity, and generalization.

Evaluation metrics:

- The application provides a clear, visual representation of where predictions match (diagonal) or are misrouted in the same evaluation section by rendering a confusion matrix calculated from the labelled examples in `classify/seed.jsonl`. In addition to the heatmap, it automatically computes macro/weighted summaries and per-class precision, recall, and F1 scores. As we add more seed data or retrain the model, we can immediately identify strong classifications, diagnose confusions (such as Billing and Delivery), and monitor improvements. Below are the screenshots of the confusion metrics





Advance improvements:

Implementing **end-to-end language support** and systematic prompt caching is a useful way to advance the application. Prior to executing RAG over the curated knowledge base, a multilingual pipeline would automatically identify the user's language, convert the query into a pivot language for retrieval, and then translate the grounded answer back into the user's language. By organizing retrieval in a single embedding space, this solution allows consistent evaluation, increases accessibility, and maintains policy fidelity across geographies. Second, by using **prompt/result caching**, RAG inputs and outputs for repeated or semantically similar questions would be memorized, thereby lowering compute costs and latency during periods of high demand. Explicit invalidation strategies should be included in the cache to guarantee accuracy as policies change.

Key files:

app/main.py ---FastAPI endpoint; defines /health, /classify, /answer, /feedback endpoints and wires classifiers + generators.

classify/seed.jsonl --- Labeled training examples for the traditional classifier (one JSON per line: {"text","label"}).

classify/traditional.py ---TF-IDF + Logistic Regression implementation,load/predict helpers and training utilities.

classify/model.joblib, classify/vectorizer.joblib ---Saved sklearn classifier and TF-IDF vectorizer artifacts used at runtime.

classify/llm.py --- LLM-based classifier (prompted JSON via HF/OpenAI/Ollama) with robust JSON/keyword fallbacks.

llm/generate.py ---Answer generator for Baseline/RAG/CoT; routes to chosen LLM provider, deterministic decoding, returns answer + retrieved docs.

prompts/answer_baseline.txt --- Prompt template for unguided (no retrieval) customer answer.

prompts/answer_rag.txt--- Prompt template that instructs using ONLY retrieved policy snippets.

prompts/answer_cot.txt--- Prompt template with brief hidden reasoning (Chain-of-Thought) over retrieved snippets.

rag/index.py--- Builds FAISS index from `data/company` and `data/public` markdown, writes `store.faiss` and `store.meta.json`.

rag/retriever.py--- Loads FAISS + embeddings, retrieves top-k passages, cleans markdown, returns text + IDs + scores.

rag/store.faiss, rag/store.meta.json ---Vector index and metadata (id→text mapping) for retrieval.

data/company/*.md --- Original company policies/SOPs (returns, delivery, privacy, deletion, incident response, etc.) used for grounding. data/public/*.md --- Seed customer reviews/complaints for retrieval realism (optionally expanded with HF/Kaggle but not used).

feedback/store.py ---Simple feedback logger (JSONL) and helpers for storing usefulness ratings/comments for later analysis.

ui/app.py--- Streamlit frontend: input box, Classify & Generate buttons, model/mode selectors, answer + citations display and confusion matrix

Reproducibility steps:

Open code in visual studio code:

Open terminal :

Cd /floder

python3 -m venv .venv

source .venv/bin/activate

pip install -r requirements.txt

python3 -m rag.index

Backend:

python3 -m uvicorn app.main:app --host 0.0.0.0 --port 8000

UI: (other terminal)

python3 -m streamlit run ui/app.py --server.port 8501 (redirects to the page)

References:

1. Fast API, High-performance Python web framework for APIs.

[<https://fastapi.tiangolo.com/>](

2. Streamlit, Python framework for data apps and simple web frontends.

[<https://streamlit.io/>](

3. scikit-learn ,Machine learning in Python (TF-IDF, Logistic Regression). Pedregosa, F., et al. (2011). Journal of Machine Learning Research, 12, 2825–2830. [<https://scikit-learn.org/>]

4. Hugging Face Transformers, Pretrained LLMs and text2text generation (e.g., FLAN-T5). Wolf, T., et al. (2020). Proceedings of EMNLP 2020: Systems Demonstrations.

[<https://huggingface.co/transformers/>]

5. MTEB: Amazon Reviews Multi (Dataset), Multilingual Amazon review corpus used for evaluation/seed examples.

[https://huggingface.co/datasets/mteb/amazon_reviews_multi](